

Contents

Objective	2
Materials Used	2
Methodology (Algorithm)	2
Step 1: Image Loading and Preprocessing	2
Step 2: Foreground Detection (Red Rose)	2
Step 3: Background Blurring	2
Step 4: Foreground Sharpening	2
Step 5: Combining Foreground and Background	3
Source Code	3
Results.....	5
Original Image	5
Processed Image	6
Observations	7
Conclusion.....	7

Objective

- To make the foreground (rose) clearer and more prominent.
- To blur the background, creating a depth-of-field effect.
- To implement image sharpening and smoothing using filtering techniques in Python.

Materials Used

- **Programming Language:** Python 3
- **Libraries:** OpenCV, NumPy, Matplotlib
- **Software/IDE:** VS Code
- **Input Image:** `rose3.jpg`
- **Hardware:** Personal Computer

Methodology (Algorithm)

The image processing was carried out in the following steps:

Step 1: Image Loading and Preprocessing

- The image was read using `cv2.imread()` and converted from BGR to RGB format using `cv2.cvtColor()`.
- The height, width, and number of channels of the image were determined and stored for further processing.

Step 2: Foreground Detection (Red Rose)

- A binary mask was created to identify the red regions corresponding to the rose.
- Pixels were classified as foreground if the red component was greater than 150, the green component was less than 100, and the blue component was less than 100.
- The resulting mask contained a value of 255 for foreground pixels and 0 for background pixels.

Step 3: Background Blurring

- A averaging filter (low-pass filter) with a kernel size of 5×5 was applied to smooth the image.
- Padding was added to handle the edges of the image during filtering.

Step 4: Foreground Sharpening

- A sharpening mask was computed by subtracting the blurred image from the original image ($\text{mask} = \text{original} - \text{blurred}$).

- The mask was added back to the original image to obtain a sharpened version of the foreground.
- Pixel values were clipped to the range of 0–255 to maintain valid intensity levels.

Step 5: Combining Foreground and Background

- The binary mask was used to merge the sharpened foreground with the blurred background.
- Foreground pixels were taken from the sharpened image, while background pixels were taken from the blurred image.
- The final output image emphasized the rose while softening the surrounding regions.

Source Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread(r"D:\3-2\Digital Image Processing\LAB\assignment_1\rose3.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
h, w, ch = img.shape
red_rose = np.zeros((h, w), dtype=np.uint8)

for x in range(h):
    for y in range(w):
        r, g, b = img[x, y]
        if r>150 and g<100 and b<100:
            red_rose[x, y] = 255
        else:
            red_rose[x, y] = 0

kernel = 5
pad = kernel//2

padded_img = np.zeros((h + 2*pad, w + 2*pad, 3), dtype=int)
padded_img[pad: pad+h, pad:pad+w] = img

blur_img = np.zeros_like(img)
for x in range(h):
    for y in range(w):
        for c in range(3):
            r = padded_img[x:x+kernel, y:y+kernel, c]
            total = 0
            for i in range(kernel):
                for j in range(kernel):
```

```

        total+=r[i, j]
    blur_img[x, y, c] = total // (karnel*karnel)
sharp_mask = np.zeros_like(img, dtype=int)
sharp = np.zeros_like(img, dtype=int)
for x in range(h):
    for y in range(w):
        for c in range(3):
            sharp_mask[x, y, c] = int(img[x, y, c]) - int(blur_img[x, y, c])
            sharp[x, y, c] = int(img[x, y, c]) + int(sharp_mask[x, y, c])
            sharp[x, y, c] = max(0, min(255, sharp[x, y, c]))
rose = np.zeros_like(img)
background = np.zeros_like(img)
for x in range(h):
    for y in range(w):
        for c in range(3):
            if red_rose[x, y]==255:
                rose[x, y, c] = sharp[x, y, c]
            else:
                background[x, y, c] = blur_img[x, y, c]
output = rose + background
plt.figure(figsize=(12, 6))
plt.subplot(1,2,1)
plt.imshow(img)
plt.title("Original")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(output)
plt.title("Manual Sharpened Foreground + Blurred Background")
plt.axis("off")

plt.show()

```

Results

Original Image

Original



Processed Image

After Transformation



Observations

- The rose appears more prominent after sharpening.
- The background is effectively blurred, providing visual contrast.
- Filtering demonstrates how spatial domain operations can enhance specific image regions.

Conclusion

- Foreground sharpening and background blurring can effectively draw attention to the main subject in an image.
- Using masks and spatial filters in Python allows selective processing of different regions.
- This assignment illustrates basic image enhancement and mask-based segmentation techniques in digital image processing.